



Research-Oriented University System

Object-Oriented Programming — Java Project

Group: Totally Spies!

Team Members

Sabit Amina
Turkpen Adiya
Nurbek Anzara

Field	Value
Discipline	Object-Oriented Programming in Java
University	Kazakh-British Technical University (KBTU)
Group	Totally Spies!
Academic Year	2025–2026
Project Parts	Part A — Diagrams Part B — Classes & Implementation
Tools	VS Code, StarUML, GitHub
Language	Java (pure OOP, no frameworks)

1. Project Overview

The Research-Oriented University System (ROUS) is a console-based Java application that models the internal operations of a university. It provides role-specific functionality for five types of users: Student, Teacher, Manager, Admin, and Researcher. The system enforces real academic rules and demonstrates the full range of OOP techniques studied in the course.

1.1 Main Goals

- Design a clean multi-level class hierarchy using abstract classes and interfaces
- Implement five user roles with distinct menus and responsibilities
- Build a Research module: papers (IEEE fields), projects, h-index validation
- Apply three design patterns: Singleton, Factory, Strategy
- Enforce academic constraints: max 21 credits, max 3 fails, supervisor h-index ≥ 3
- Implement serialization for persistent data storage across runs
- Produce four UML diagrams: Object Diagram, Use Case, Class (before), Class (after patterns)

2. System Architecture & Package Structure

The project is organized into seven packages ensuring low coupling and high cohesion.

Package	Key Classes	Responsibility
model.user	User, Employee, Admin, Manager, Teacher, Professor, Tutor, SeniorLecturer, Student	All user roles and hierarchy
model.academic	Course, Lesson, Mark, Transcript, CourseRegistration, TeacherRating, Message, News	Academic process and communication
model.research	Researcher (interface), ResearchPaper, ResearchProject	Research module
pattern	Database, UserFactory, AuthService, UserSession	Singleton, Factory, Auth
strategy	SortStrategy, SortByDateStrategy, SortByCitationsStrategy, SortByLengthStrategy	Strategy pattern for sorting papers
enums	Role, TeacherTitle, LessonType, ManagerType, RegistrationStatus, RiskLevel, ScholarshipStatus	All enumerations
exception	LowHIndexException, NonResearcherJoinException, CreditLimitException, FailLimitException, UserNotFoundException, CourseFullException	Custom checked exceptions

3. Class Descriptions

3.1 User Hierarchy

Field	Value
User (abstract)	Core identity: userId, login, passwordHash, role, isActive. Methods: checkPassword(), getInfo() [abstract], equals(), hashCode(), toString()
Employee (abstract)	Extends User. Adds: firstName, lastName, email, salary, department, inbox. Methods: sendMessage(), receiveMessage()
Admin	Extends Employee. addUser(), removeUser(), resetPassword(), banUser(), viewLogs()
Manager	Extends Employee. managerType (enum). approveRegistration(), rejectRegistration(), assignTeacherToCourse(), generateReport(), publishNews()
Teacher (abstract)	Extends Employee. Abstract: isResearcher(). Methods: putMark(), viewStudents(), getAverageRating()
Professor	Extends Teacher, implements Researcher. Always a researcher. hIndex field. addSupervisedStudent() throws LowHIndexException if hIndex < 3
Tutor	Extends Teacher, implements Researcher optionally
SeniorLecturer	Extends Teacher, implements Researcher optionally
Student	Extends User, implements Comparable<Student> by GPA. yearOfStudy, gpa, credits (max 21), failCount (max 3), transcript, supervisor, riskLevel

3.2 Academic Classes

Field	Value
Mark	$att1*0.3 + att2*0.3 + finalExam*0.4 = total$. isFail(): total < 50. getLetterGrade(): A/B/C/D/E/F
Transcript	Composition with Student (1:1). getGPA() normalizes to 4.0 scale. formatTranscript() renders boxed academic record
Course	courseId, courseName, credits, maxStudents, targetYear, List<Teacher> instructors (multiple allowed)
Lesson	topic, lessonType (LECTURE/PRACTICE), date, durationMinutes
CourseRegistration	PENDING -> APPROVED/REJECTED. approve(Manager) enrolls student
Message / News	Employee-to-employee messaging; News published by Manager via Observer pattern

3.3 Research Classes

Field	Value
-------	-------

Researcher (interface)	getHIndex(), printPapers(Comparator), addPaper(), addProject(), getTotalCitations(). Implemented by Professor always; others optionally
ResearchPaper	IEEE fields: title, authors, journal, doi, publicationDate, citations, pages, keywords, abstractText, isOpenAccess. Implements Comparable by citations. equals/hashCode by DOI
ResearchProject	addParticipant(User) throws NonResearcherJoinException if not instanceof Researcher

4. OOP Concepts Applied

Concept	Where Applied	Detail
Inheritance	User -> Employee -> Teacher -> Professor	4-level hierarchy, 9 concrete classes
Abstraction	User, Employee, Teacher — abstract	Cannot be instantiated; enforce contracts
Interface	Researcher, SortStrategy	Role contract independent of class hierarchy
Polymorphism	printPapers(Comparator c)	Same call, different sort behavior
Encapsulation	All fields private	Access strictly through getters/setters
Comparable<T>	Student (by GPA), ResearchPaper (by citations)	Natural ordering for sort
Comparator<T>	3 Strategy classes	External ordering without modifying source
equals/hashCode	Student (userId), ResearchPaper (doi)	Correct behavior in collections
Stream API	Database queries, Manager reports	filter(), sorted(), mapToInt()
Serialization	Database.save() / load()	All model classes implement Serializable
Enumerations	7 enums	Type-safe state representation
Custom Exceptions	6 checked exceptions	Domain-specific error handling

5. Design Patterns

Pattern	Class	Motivation	How It Works
Singleton	Database	One central data store must exist system-wide.	Private constructor. Static getInstance(). All classes use Database.getInstance().
Factory	UserFactory	Admin creates users without knowing which concrete class.	create(Role, ...) switch-cases on Role and returns correct subclass.
Strategy	SortStrategy + 3 classes	printPapers() must sort by different criteria without changing Professor.	SortStrategy interface with sort(). Three implementations: ByDate, ByCitations, ByLength.

6. Code Fragments

6.1 User — abstract class, auth

User.java

```
public abstract class User implements Serializable {
    private final String userId;
    private String login;
    private String passwordHash;
    private Role    role;
    private boolean isActive;

    public boolean checkPassword(String raw) {
        return passwordHash.equals(Integer.toHexString(raw.hashCode()));
    }
    public abstract String getInfo();
    @Override public boolean equals(Object o) {
        if (!(o instanceof User)) return false;
        return Objects.equals(userId, ((User) o).userId);
    }
}
```

6.2 Mark — grade calculation

Mark.java

```
public double getTotal() {
    return attestation1*0.3 + attestation2*0.3 + finalExam*0.4;
}
public boolean isFail()          { return getTotal() < 50.0; }
public String getLetterGrade() {
    double t = getTotal();
    if (t >= 90) return "A";  if (t >= 80) return "B";
    if (t >= 70) return "C";  if (t >= 60) return "D";
    if (t >= 50) return "E";  return "F";
}
```

6.3 Student — registration with constraints

Student.java

```
public CourseRegistration registerForCourse(Course course)
    throws CreditLimitException, FailLimitException {
    if (failCount >= MAX_FAILS)
        throw new FailLimitException(failCount);
    if (credits + course.getCredits() > MAX_CREDITS)
        throw new CreditLimitException(credits, course.getCredits(),
MAX_CREDITS);
    CourseRegistration reg = new CourseRegistration(this, course);
    registrations.add(reg);
    return reg;
}
```

6.4 Researcher interface + Professor

Researcher.java (interface)

```
public interface Researcher {
    int getHIndex();
    void printPapers(Comparator<ResearchPaper> c);
    void addPaper(ResearchPaper paper);
    int getTotalCitations();
}
```

Professor.java — implements Researcher

```
public class Professor extends Teacher implements Researcher {
    @Override
    public void printPapers(Comparator<ResearchPaper> c) {
        researchPapers.stream().sorted(c).forEach(System.out::println);
    }
    @Override
    public int getTotalCitations() {
        return researchPapers.stream()
            .mapToInt(ResearchPaper::getCitations).sum();
    }
}
```

6.5 ResearchProject — exception on join

ResearchProject.java

```
public void addParticipant(User user) throws NonResearcherJoinException {
    if (!(user instanceof Researcher))
        throw new NonResearcherJoinException(user.getLogin());
    participants.add((Researcher) user);
}
```

6.6 Database — Singleton pattern

Database.java

```
public class Database {
    private static Database instance;
    private Database() {}
    public static Database getInstance() {
        if (instance == null) instance = new Database();
        return instance;
    }
    public List<Student> getAllStudents() {
        return users.values().stream()
            .filter(u -> u instanceof Student)
            .map(u -> (Student) u)
            .collect(Collectors.toList());
    }
}
```

6.7 Strategy — SortStrategy

SortStrategy.java + SortByCitationsStrategy.java

```
public interface SortStrategy {
    List<ResearchPaper> sort(List<ResearchPaper> papers);
}

public class SortByCitationsStrategy implements SortStrategy {
```

```

@Override
public List<ResearchPaper> sort(List<ResearchPaper> papers) {
    return papers.stream()

.sorted(Comparator.comparingInt(ResearchPaper::getCitations).reversed())
    .collect(Collectors.toList());
}

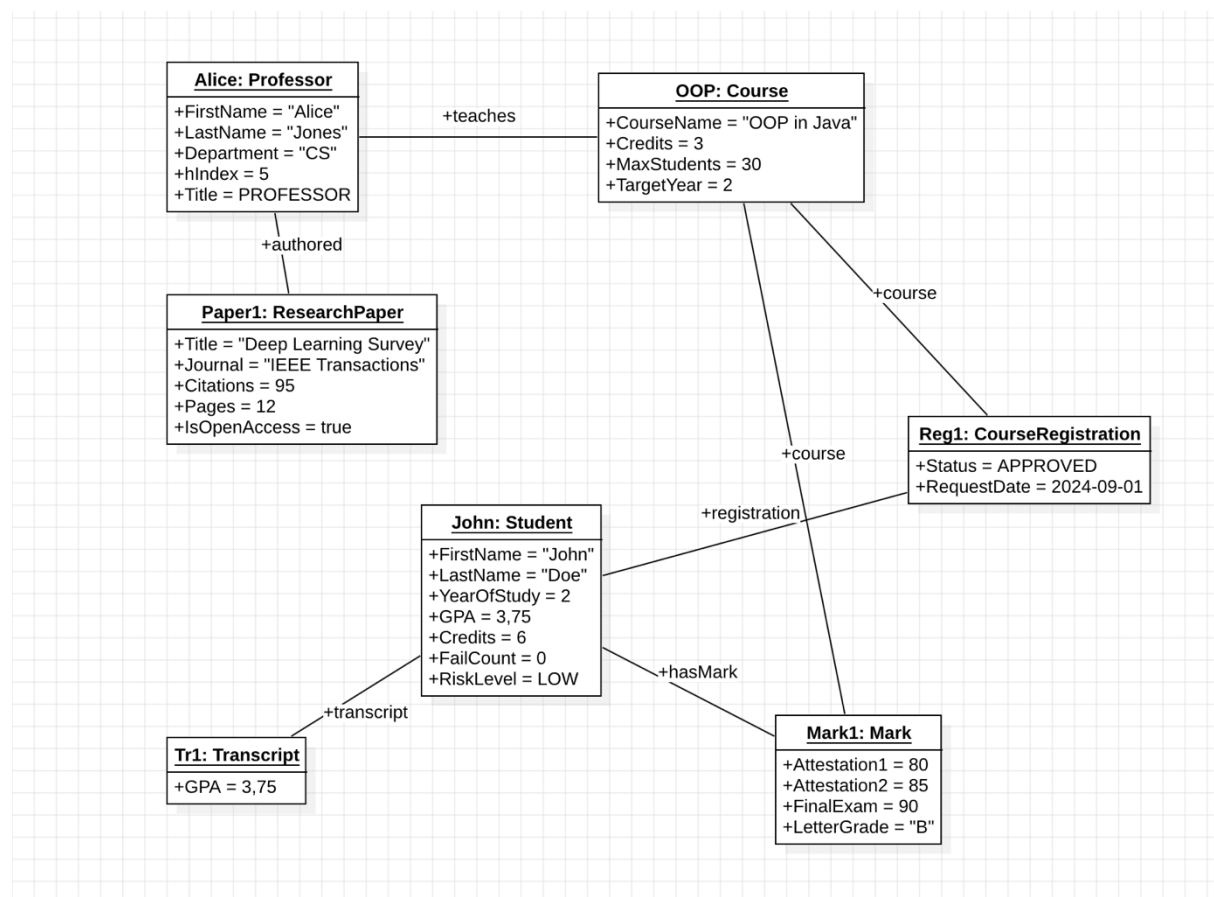
// Usage:
prof.printPapers(new SortByCitationsComparator());
prof.printPapers(new SortByDateComparator());

```

7. UML Diagrams

All four diagrams were created in StarUML. The .mdj project file is included in the submission.

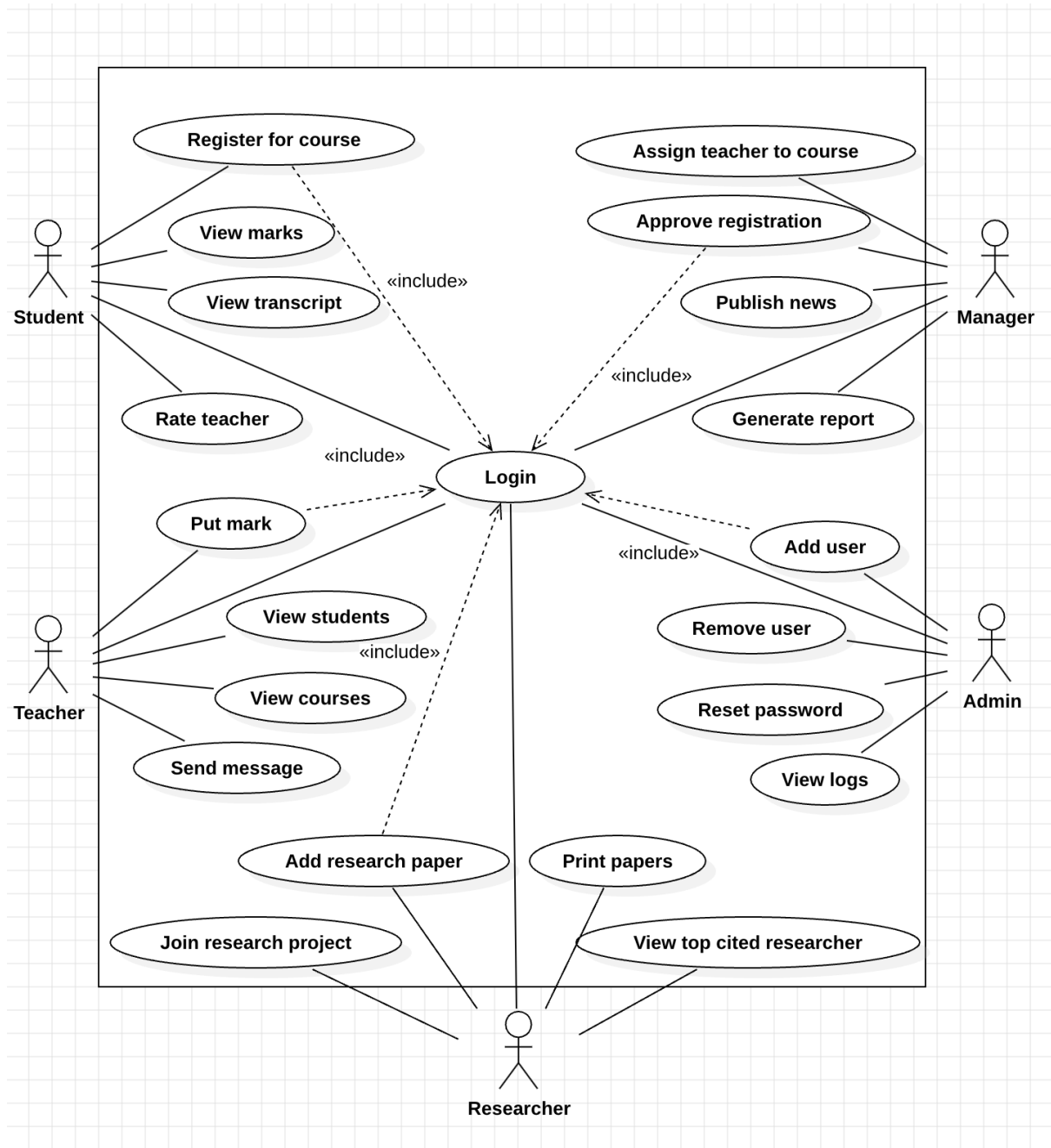
7.1 Object Diagram



Object	Class	Key Slot Values
john	Student	firstName="John", yearOfStudy=2, gpa=3.75, riskLevel=LOW
alice	Professor	firstName="Alice", department="CS", hIndex=5, title=PROFESSOR
oop	Course	courseName="OOP in Java", credits=3, maxStudents=30

mark1	Mark	attestation1=80, attestation2=85, finalExam=90, grade="B"
tr1	Transcript	gpa=3.75
reg1	CourseRegistration	status=APPROVED, requestDate=2024-09-01
paper1	ResearchPaper	title="Deep Learning Survey", citations=95, pages=12

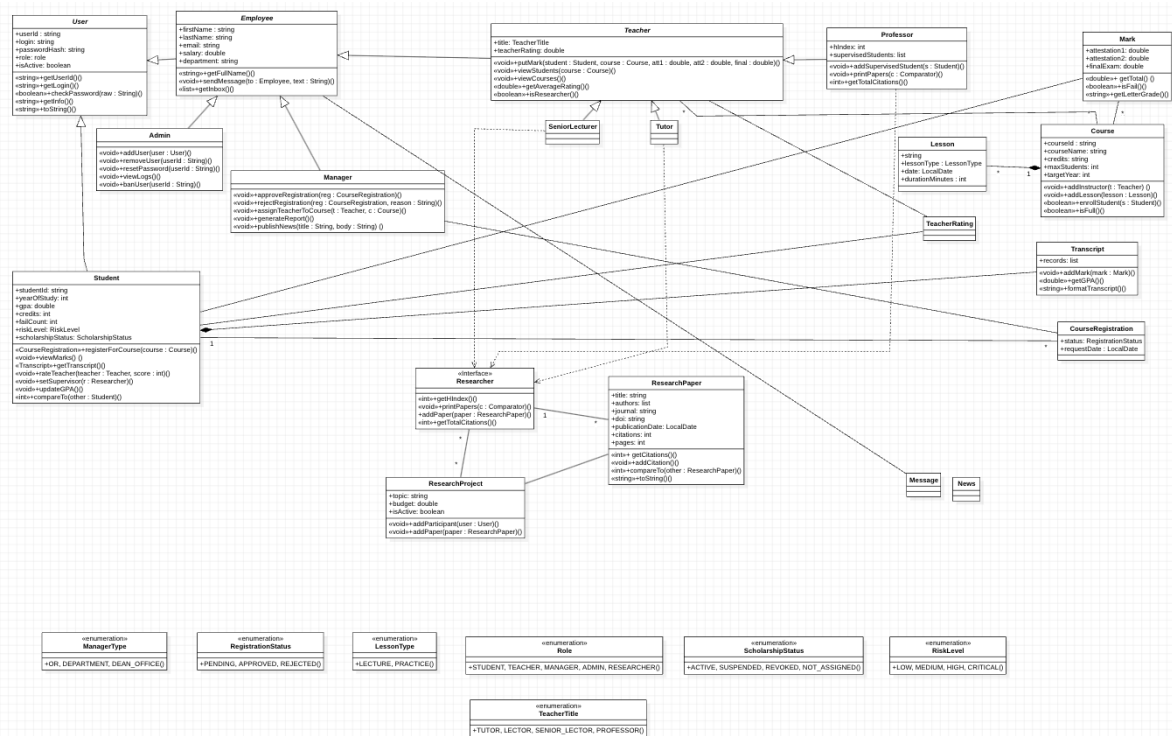
7.2 Use Case Diagram



Actor	Use Cases
Student	Register for Course, View Marks, View Transcript, Rate Teacher

Teacher	Put Mark, View Students, View Courses, Send Message
Manager	Approve Registration, Assign Teacher, Publish News, Generate Report
Admin	Add User, Remove User, Reset Password, View Logs, Ban User
Researcher	Add Research Paper, Join Research Project, Print Papers, View Top Cited

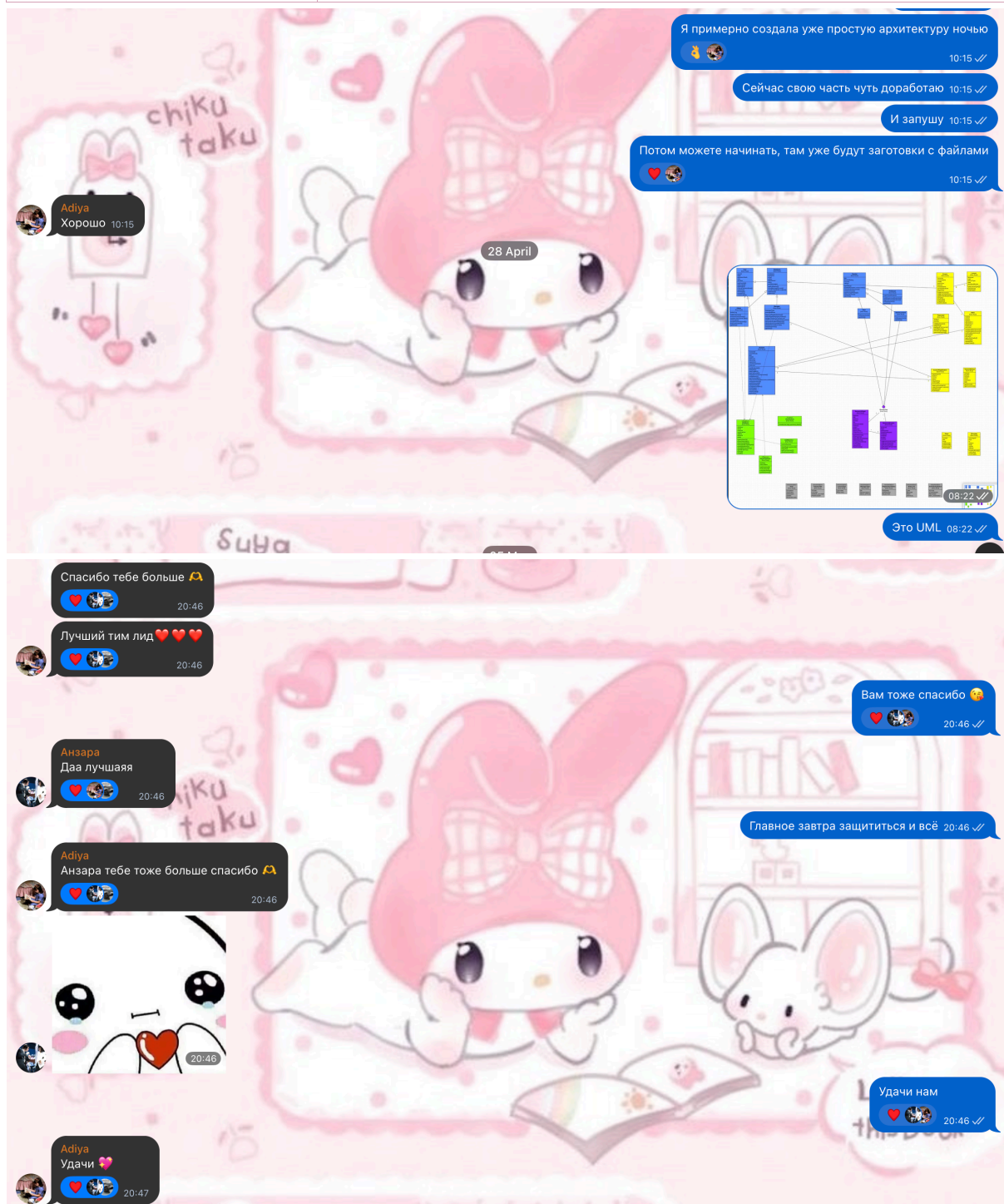
7.3 Class Diagram — Before Patterns



- Generalization (>): Employee->User, Admin/Manager/Teacher->Employee, Professor/Tutor/SeniorLecturer->Teacher, Student->User
- Realization (-->): Professor, Tutor, SeniorLecturer -> Researcher interface
- Composition: Student-Transcript, Course-Lesson
- Association: Student-CourseRegistration (1:*), Course-Mark (1:*), Researcher-ResearchPaper (1:*)

7.4 Class Diagram — After Patterns

Turkpen Adiya	Teacher hierarchy (Teacher, Professor, Tutor, SeniorLecturer), Student, all academic classes (Course, Lesson, Mark, Transcript, CourseRegistration, Message, News)
Nurbek Anzara	Research module (Researcher, ResearchPaper, ResearchProject), Strategy pattern, Main.java interactive menu, README, StarUML diagrams



(some of our telegram group conversations)

9.2 Timeline

- Day 1 — Foundation: enums, exceptions, User/Employee/Teacher skeletons, Researcher interface, GitHub structure
- Day 2 — Core logic: Student constraints, Mark calculation, ResearchProject validation, Database Singleton
- Day 3 — Integration: Strategy pattern, interactive console menu, StarUML diagrams, README, serialization

9.3 Tools

- VS Code — primary IDE for all three developers
- GitHub — shared repository, incremental commits per feature
- StarUML — all four UML diagrams (.mdj file included)
- Telegram — team coordination and code review

9.4 Challenges

- Researcher as interface vs abstract class: chosen as interface because Professor already extends Teacher. Java has no multiple class inheritance. Allows Professor, Student, and Employee to all be Researchers independently.
- Serialization with circular references: Student->Transcript->Mark->Student required all classes to implement Serializable carefully.
- Package naming conflict: all source files had incorrect 'src.com.university' package declarations which required a global sed fix before compilation succeeded.

10. Conclusion

The Research-Oriented University System implements all required classes from the task specification — User, Employee, Teacher, Manager, Student, Admin, Course, Mark, Lesson, Researcher, ResearchPaper, ResearchProject — plus 12 supporting classes across 7 packages.

Three design patterns (Singleton, Factory, Strategy) are applied with clear motivation. All academic business rules are enforced via custom checked exceptions. The project includes four UML diagrams, serialization, JavaDoc documentation, and a fully interactive console menu covering all five user roles.